

# Opdracht 1 - YAML-JSON-server

## Beschrijving

Je ontwikkelt een toepassing vergelijkbaar met json-server, waarbij data wordt geconfigureerd via een YAML-bestand (zie <https://github.com/typicode/json-server/>). De toepassing draait lokaal op `http://localhost:3000/` en ondersteunt dynamische routes, CRUD-operaties, en relaties tussen routes. Je werk wordt bijgehouden in een GitHub-repository (zie Blackboard).

Commit en push je werk regelmatig naar de repository, idealiter na elke significante wijziging of na werksessies van ongeveer 15 minuten. Toon je vorderingen via een progress-report en verdedig het eindproduct. Deadlines zijn te vinden op Blackboard. Een basisimplementatie is beschikbaar op Blackboard, maar je mag hiervan afwijken.

Een deel van de code kan je al terugvinden op BB (je mag hiervan afwijken).

## Configuratie

Het YAML-bestand bevat altijd:

- Een lijst van routes
- Per route een array van objecten (records), elk met een uniek id (integer).

Voorbeeld met relaties:

```
routes:
  - persons
  - pets

relationships:
  pets:
    - relatedRoute: persons
      foreignKey: ownerIds

persons:
  - id: 1
    name: Jan
  - id: 2
    name: Sofie
  - id: 3
    name: Tim

pets:
  - id: 1
    type: dog
    name: Jimmy
    ownerIds: [2]
  - id: 2
    type: cat
    name: Duvel
    ownerIds: [1, 2]
```

Eenvoudiger voorbeeld:

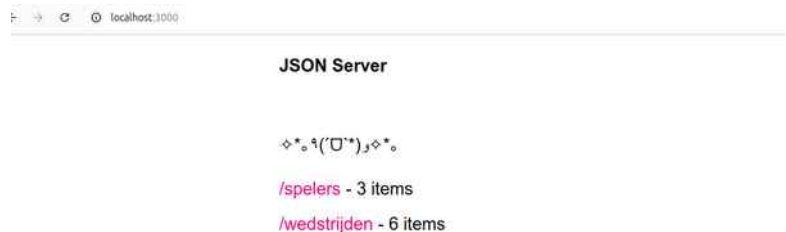
```
routes:
  - personen

personen:
  - id: 1
    naam: Jan
```

## Functionaliteiten

### 1. Landing page

- Gebruik een **templating engine** voor de startpagina (<http://localhost:3000/>).
- Toon links naar alle beschikbare routes, zodat gebruikers GET-requests kunnen uitvoeren (zoals in json-server). Ter inspiratie kan je hieronder de landing-page van json-server vinden.



### 2. GET-requests

- GET /<route> → retourneer alle records van de route (gegeven).
- GET /<route>/<id> → retourneer één record op basis van id (gegeven).
- **Embedded relaties:**  
Als een embed-parameter wordt meegegeven, bijvoorbeeld:
  - GET /pets?embed=owners
  - GET /pets/1?embed=owners

Dan wordt de gerelateerde data toegevoegd aan het record:

```
{
  "id": 1,
  "type": "dog",
  "name": "Jimmy",
  "ownerIds": [2],
  "owners": [
    {
      "id": 2,
      "name": "Sofie"
    }
  ]
}
```

### 3. Mutaties

- POST /<route> → voeg een nieuw record toe.
- PUT /<route>/<id> → update een bestaand record.
- DELETE /<route>/<id> → verwijder een record.

### 4. Persistente toestand

- De nieuwe toestand wordt tijdens runtime in **geheugen** bewaard.
- Bij afsluiten van de server wordt de bijgewerkte toestand teruggeschreven naar het YAML-bestand.

**Als je de opgave zoals hierboven uitwerkt verdien je 12/20**

### Aandachtspunten

- per 15 minuten werk push je naar de online repo (geen commit history, geen punten)
- je kan vragen over je code beantwoorden tijdens progress report en eindverdediging
- tijdens de verdedigingen wordt een stuk code weggeknipt en moet je de code kunnen herstellen (zonder ctrl-z, zonder git reset --hard HEAD 😊 )
- geen ctrl c-ctrl v uit chatgpt, grok, ... (je mag wel informatie opzoeken)

# Uitbreidingen

## 1. E2E-test (+1pt)

- Gebruik bijvoorbeeld **Supertest** om de volledige API te testen (GET, POST, PUT, DELETE, embed).

## 2. UUIDs in plaats van integers (+1pt)

- Gebruik uuid voor de id's (ipv int).

## 3. Opsplitsing (+1pt)

- app.js → Express setup, config uitlezen, routes registreren.
- service.js (of een andere naam) → data (in het geheugen) opzoeken, wijzigen

## 4. Uitgebreid product (per extra feature +1pt)

- het product kan qua functionaliteit wedijveren met json-server  
<https://github.com/typicode/json-server>  
(range, paginate, sort, ...)